



[BIT703 Web Technologies](#) / [Contents](#) / [Topic 11: Server-side web application admin panel development](#) /
Authorisation and authentication

Maintaining a logged-in state 20m

In previous topics, the web page examples have shown individual HTTP/S request/response cycles between a client browser and a web page. In these interactions, as soon as the HTTP/S response is sent back, the link between the client and the server is lost. This is because HTTP/S is a stateless protocol, meaning it is not designed to maintain the connection state between requests and responses between the browser or client and the web server. Consequently, the server is unable to recognise if multiple requests coming from a single client are part of a linked transaction.

However, in web applications that gather client data, a solution is required that will be able to recognise the same client over multiple request/response cycles so that, for example, a client using an online store can start shopping, browse through a catalogue, add items to their shopping cart (or remove them) and eventually purchase all the items in the cart, regardless of how many pages they have viewed in the meantime. For this to work, there needs to be some way of identifying the individual client so that everyone has access to their own, and only their own, shopping cart, streaming account, bank account and so on.

A web application can maintain this information on the server in a session object, a separate instance of which will be created for each client using the web application. A session represents a conversation between a client and a server that includes multiple request/response cycles.

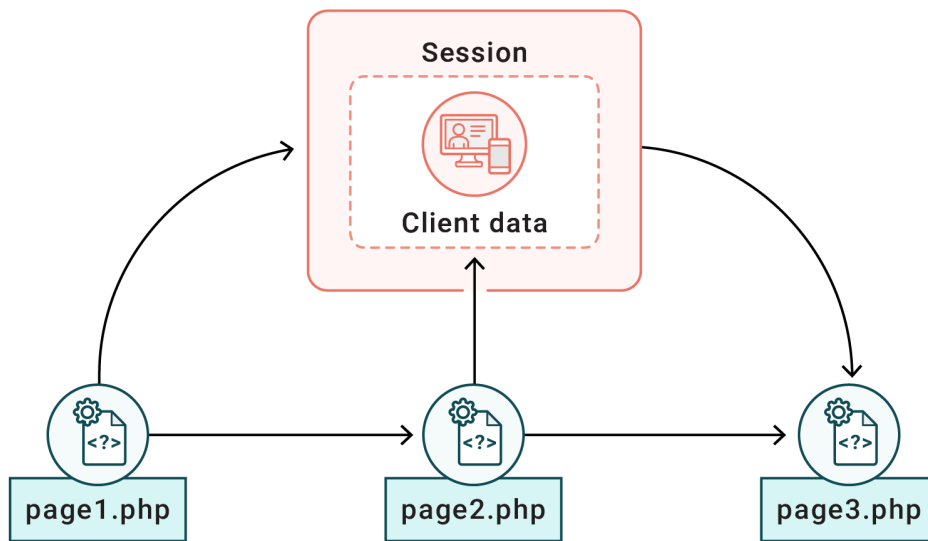
Sessions

A session is when the client needs to maintain some state over a series of different web pages. The image below depicts a session which is used to maintain data over different web pages.

[Skip to main content](#)



API session



© © David Parsons



A common example of needing to maintain a client state is to keep track of whether the user is logged in as they navigate a website, enabling their access to resources on the server to be authorised appropriately. This is the scenario that will be used for the remaining code examples in this topic.

[Skip to main content](#)

Cookies and sessions

Sessions normally use cookies to maintain the identity of the client. A cookie is a name/value pair that can be sent from the server to the client in an HTTP response. The cookie is sent by the browser with any further requests to the same URL so the server can recognise that the client has connected previously. When a session is created, a session cookie is sent back with the next response to the client, and that session cookie is then passed along with subsequent requests and responses. The server can use this cookie to identify which session should be associated with the client. In PHP, if the cookies are disabled the sessions will still work just with the session reference passed as part of the request URL.

In terms of its implementation, a session is a collection of key-value pairs that can be used to keep track of application-specific information about clients. When you create a session in PHP, it will also create the global variable `$_SESSION`, which will survive page reloads and browsing to different pages within the web application. It is inside this object that application-specific key-value pairs can be stored.

It should also be noted when working with sessions that you should also be including the mechanism to handle CSRF (cross-site request forgery) for the client and the server. You can read through this PHP CSRF article (PHP Tutorial, n.d.) to see how the sessions are used to maintain the CSRF tokens. For the following example, CSRF has not been implemented, but the process as per the above article is the same as indicated below just with an additional session variable.

PHP sessions

To keep track of a user who is logged in to the admin panel, their details need to be kept in a session while they are logged in. The **`session_start`** function is used to create or access a session in PHP.

```
session_start()
```

This function must be the first thing included in a PHP file. Once the session has been created, session variables (consisting of a key and value) can be set using the PHP global variable `$_SESSION`.

[Skip to main content](#)

The following example shows how a PHP page that creates a session might begin, setting up a session with variables related to a client's login status. Initially, in this example, the session contains no login data – 'logged in' is set to 'false' and the 'email' is set to an empty string.

The email is, of course, used to identify the user. This initialisation will be done on the login page when the user first logs in to the admin panel.



Copy

```
1  <?php
2  session_start();
3  $_SESSION['loggedin'] = false;
4  $_SESSION['email'] = '';
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

At the point where the user's email and password are validated against the database, their session can be populated with data.

Note that any page that interacts with sessions needs to have the **session_start** function at the top, since without it, the `$_SESSION` object cannot be accessed.



[Skip to main content](#)

Thoughts on this page?

